opportunity to test the process's behavior during the test rather than the outcome.

Often, this kind of test makes sense given a relatively complex set of object interactions. You don't exactly have that here, but you can use your controller as a reasonable stand-in. Some features of controller tests are deprecated in Rails 5, but you won't touch any of the deprecated features and you'll test the controller as its own unit, without dependencies on any other code. I think using test doubles is still a potentially valid way of unit-testing logic in controllers.

Traditional controller tests—the deprecated kind—often test whether the controller is setting a particular instance variable to pass off to the view and giving it a value matching the incoming data. However, given the way you've written the controller, the specific values are the workflow's responsibility. The controller is acting as merely a conduit. Its job is to get the value from some data source and pass it on. Using the controller test to verify the workflow or view behavior leads to brittle tests.

What is the controller's responsibility? The controller is just a traffic cop. If you test that the controller sets an instance variable, you're testing the output of the conduit. The input of the conduit, though, is verified not by the state of the variable at the end of the controller call, but by the way that value is obtained. In other words, the controller's responsibility is to meet a contract on both ends—it's responsible for setting a particular value to satisfy the view, and it's responsible for calling some other place in the system to acquire data.

Specifically, the controller calls `CreatesProject.new` and then calls `create` on the resulting action. Leave the responsibility of verifying that `CreatesProject` accurately creates a project to the tests for that object (tests you wrote when you added the `CreatesProject` functionality). All you need to do here is specify that the controller calls the appropriate methods.

Enter mocks. Mock objects are used to set an expectation for the controller's behavior during the user action. This will look similar to the failure test from the last section, but I'm going to emphasize a different part of the test:

mocks/01/spec/controllers/projects_controller_spec.rb

```
require "rails_helper"

RSpec.describe ProjectsController, type: :controller do

  describe "create" do
    it "calls the workflow with parameters" do
      workflow = instance_spy(CreatesProject, success?: true)
      allow(CreatesProject).to receive(:new).and_return(workflow)
```